Proje 2: Gerçek Zamanlı Veri Akışı ve İşleme (IoT veya WebSocket Uygulaması)

BLM3522: Bulut Bilişim ve Uygulamaları

Öğrenci	Aura Nawamadanı Amrah
Proje Adı	Eattheapple
GitHub	https://github.com/Bananaurawr/eattheapple
Video URL	Eattheapple.mp4
Tarih	18.04.2026

İçerikler

Giriş..... 3
Kullanılan Teknolojileri..... 4
Frontend..... 4
Backend..... 5
Bulut Teknolojileri..... 5
Backend Yapısı..... 6
Kullanılan Kütüphaneleri..... 6
Google Pub/Sub Setup..... 6
BigQuery Setup..... 7
Flask Routes and SocketIO events..... 8
Analizi..... 9

Giriş

Önceki projede, Google Cloud Hizmetlerini kullanarak temel bir forum sitesi oluşturdum; bu projede web hosting için Firebase'i, backend hosting için Google Cloud Run'ı ve kullanıcı veritabanı için Google Cloud SQL'i kullandım. Bu proje sayesinde, özellikle yorum bölümünde canlı oylama sistemi ya da her kullanıcının en sık kullandığı kelimelerin analizini içeren bir canlı sohbet gibi gerçek zamanlı analiz özelliklerini entegre etmenin birçok yolu var.

Web sitesinde backend mimarisi, sistem mimarisi ve frontend açısından hala geliştirebileceğim birçok alan var. Bu nedenle, bu Eattheapple web sitesi, önceki projenin geçici bir uzantısı olarak işlev görüyor; burada kullanıcılar gecikme olmadan gerçek zamanlı olarak sohbet edebilir ve backend'de sohbet odasında gönderilen sohbetlerin analizini görebilirsiniz.

Eatthetapple'ı AUForum'un bir uzantısı olarak oluşturmanın amacı, öğrencilerin sosyal yaşamını zenginleştirmiştir; bu sayede (gelecekte) hesabı olan kişiler, ilgilendikleri konular hakkında birbirleriyle gerçek zamanlı olarak mesajlaşabileceklerdir. Ancak bu sürüm bir demo sürümü olduğu için, backend henüz kullanıcı adı kontrolü eklenmemiştir; bunun yerine, kullanıcı kendi kullanıcı adını yazarak sohbet odasına serbestçe girebilir.

Bu durum, online güvenlik açısından büyük bir sorun yaratmaktadır, çünkü çevrimiçi ifade özgürlüğünün kötüye kullanılmasına ve cyberbullying ile ilgili büyük sorunlara yol açabilir; bu nedenle web sitesi şimdilik herkese açık değildir. Daha önceki güncellemelerde de belirtildiği gibi, gelecekte kullanıcılar sohbet odalarına erişebilmek için önce web sitesinde oturum açmalıdır.

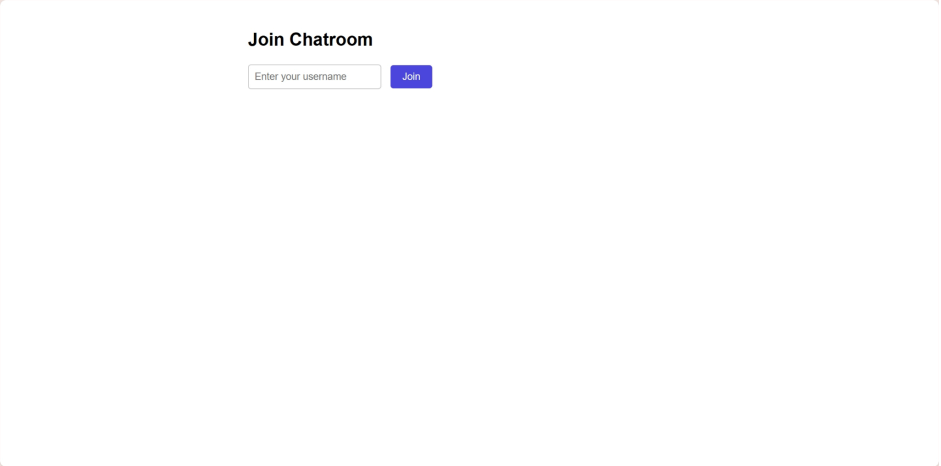
Kullanılan Teknolojileri

The project makes use of cloud technologies for data

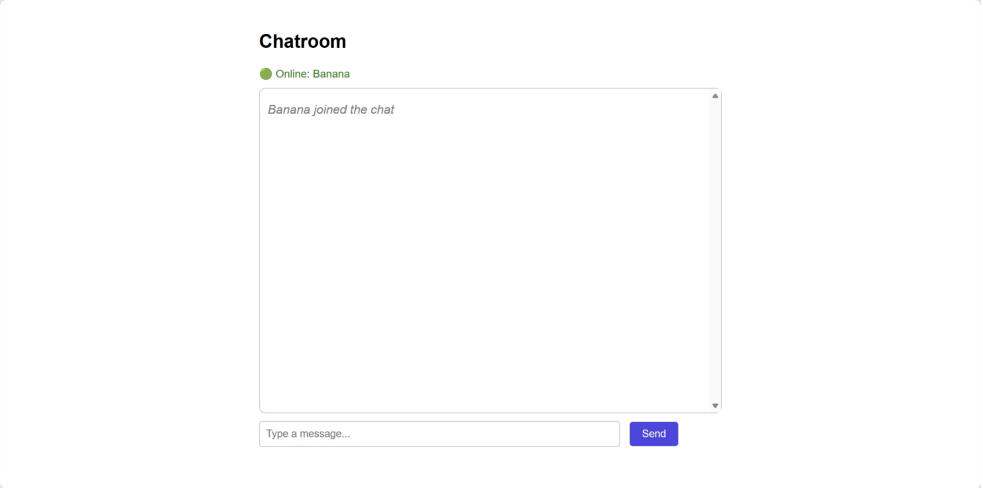
Frontend

Projenin Frontend'i sadece temel HTML ve JavaScript teknolojilerini kullanmaktadır; burada Socket kütüphanesini kullanarak web sitesinden gönderilen mesajları topluyor, kullanıcıları yönetiyor, çevrimiçi kullanıcıları kontrol ediyor ve kullanıcı adlarını işliyoruz.

Enter username (Join) page:



Chatroom page:



Backend

Backend tarafında Python programlama dili kullanılmıştır. Web sunucusu olarak Flask framework'ü tercih edilmiş, gerçek zamanlı iletişim için ise Flask-SocketIO kütüphanesi entegre edilmiştir. Flask-SocketIO, WebSocket protokolü üzerinden kullanıcılar ile sunucu arasında kesintisiz ve anlık bir bağlantı kurulmasını sağlamaktadır. Bu sayede kullanıcılar sayfayı yenilemeden birbirlerinin mesajlarını anında görebilmektedir.

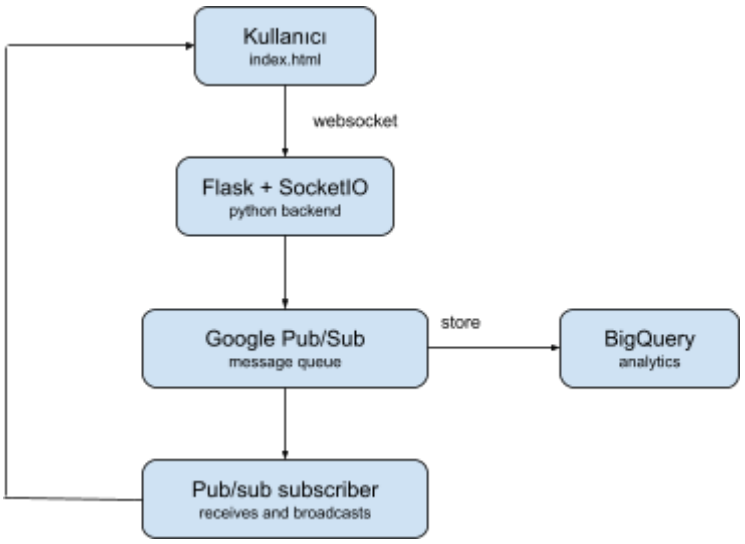
Backend’i ayrıca gönderilen mesajlar üzerinde analiz yapmamızı sağlar; localhost:5000/stats (yerel backend stats sayfası) adresini açtığımızda karşımıza şöyle bir sayfa çıkar:


```
[
  {
    "avg_words": 2.6666666666666665,
    "message_count": 3,
    "username": "Banana"
  },
  {
    "avg_words": 2.0,
    "message_count": 1,
    "username": "Aura"
  }
]
```


Bulut Teknolojileri

Bulut tarafında Google Cloud Platform kullanılmıştır. Her kullanıcı mesajı göndermekten önce Google Cloud Pub/Sub servisine iletilmektedir. Pub/Sub, mesajları bir kuyruk sistemi aracılığıyla işleyerek ilgili abonelere dağıtmaktadır. İşlenen mesajlar daha sonra Google BigQuery'e kaydedilmektedir. BigQuery üzerinde kullanıcı başına gönderilen mesaj sayısı ve ortalama kelime uzunluğu gibi analizler gerçekleştirilmektedir.

Backend Yapısı



Kullanılan Kütüphaneleri

Flask, HTTP isteklerini işleyen ve yönlendirmeyi sağlayan temel web çerçevesi olarak çalışır. Flask-SocketIO, Flask'ı WebSocket özellikleriyle genişleterek iki yönlü gerçek zamanlı iletişimi sağlar. Flask-CORS, Cross-Origin Resource Sharing'i yöneterek frontend'in farklı bir kaynaklardan backend ile iletişim kurmasını sağlar. google.cloud kütüphaneleri, Google Cloud Pub/Sub ve BigQuery hizmetleriyle etkileşim kurmak için resmi Python client'ları sağlar. Datetime zaman stamp'ları oluşturmak için kullanılır ve dotenv, .env dosyasından ortam değişkenlerini yükleyerek API anahtarları gibi hassas kimlik bilgilerini kaynak koddan uzak tutar.

```
from flask import Flask, request, jsonify
from flask_socketio import SocketIO, emit
from flask_cors import CORS
from google.cloud import pubsub_v1 ← load Google Pub/Sub library

import os
import json
import threading

from google.cloud import bigquery ← load Google BigQuery library
from datetime import datetime
```

```
from dotenv import load_dotenv ← to load credentials for Google
load_dotenv()
```

Google Pub/Sub Setup

Bu bölümde Google Cloud Pub/Sub ile bağlantı kurulur. PublisherClient, belirli bir konuya mesaj gönderme görevini üstlenirken, SubscriberClient bir aboneliği dinler ve bu mesajları alır. topic_path ve subscription_path, Google Cloud projesi içindeki belirli bir Pub/Sub konusuna ve aboneliğine işaret eden tam tanımlı kaynak tanımlayıcılarıdır.

```
# Google cloud pub/sub setup
PROJECT_ID = 'eattheapple'
TOPIC_ID = 'chat-messages'
SUBSCRIPTION_ID = 'chat-sub'

publisher = pubsub_v1.PublisherClient()
subscriber = pubsub_v1.SubscriberClient()

topic_path = publisher.topic_path(PROJECT_ID, TOPIC_ID)
subscription_path = subscriber.subscription_path(PROJECT_ID,
SUBSCRIPTION_ID)

# Store online users in memory
online_users = {}

# Publish message to pub/sub
def publish_message(username, text):
    data = json.dumps({'username': username, 'text':
text}).encode('utf-8')
    publisher.publish(topic_path,data)
    print(f"Published to Pub/Sub: {username}: {text}")
```

publish_message fonksiyonu, mesaj verilerini JSON formatına dönüştürür, bayt olarak kodlar ve Pub/Sub konusuna gönderir. Bu işlem, her sohbet mesajının bulut altyapısına giriş noktası görevi görür.

```
# Listen to Pub/Sub and broadcast to all WebSocket users
def listen_for_messages():
    def callback(message):
        data = json.loads(message.data.decode('utf-8'))
        socketio.emit('message', {
            'username': data['username'],
            'text': data['text']
        })
        message.ack()
        log_message_to_bigquery(data['username'], data['text'])
    subscriber.subscribe(subscription_path, callback=callback)
    print("Listening to Pub/Sub")

# Start Pub/Sub listener in the background
thread = threading.Thread(target=listen_for_messages)
thread.daemon = True
thread.start()
```

BigQuery Setup

```
bq_client = bigquery.Client()
TABLE_ID = 'eattheapple.chat_analytics.messages'

def log_message_to_bigquery(username, text):
    timestamp = datetime.utcnow().isoformat()
    word_count = len(text.split())
    rows_to_insert = [{
        'username': username,
        'message': text,
        'word_count': word_count,
        'timestamp': timestamp
```

```
    }]
    errors = bq_client.insert_rows_json(TABLE_ID, rows_to_insert)
    if errors:
        print(f"Error inserting into BigQuery: {errors}")
    else:
        print(f"Logged message to BigQuery: {username}: {text}")
```

Bu bölümde BigQuery client'i başlatılır ve hedef tablo tanımlanır. log_message_to_bigquery fonksiyonu, gönderenin kullanıcı adı, mesaj içeriği, kelime sayısı ve bir UTC zaman damgasını içeren bir satır oluşturur. Ardından bu satırı insert_rows_json işlevini kullanarak BigQuery tablosuna ekler. Ekleme işlemi sırasında herhangi bir hata oluşursa, hata ayıklama amacıyla bu hatalar konsola yazdırılır.

Flask Routes and SocketIO events

```
@app.route('/')
def home():
    return {"message": "Chatroom backend is running"}
```

Aşağıdaki fonksiyon, bir client WebSocket bağlantısı kurduğunda otomatik olarak çalıştırılır. Her client'e benzersiz bir oturum kimliği (request.sid) atanır.

```
@socketio.on('connect')
def handle_connect():
    print(f"Connected: {request.sid}")
```

Aşağıdaki fonksiyon, bir istemcinin bağlantısı kesildiğinde tetiklenir. Kullanıcı, `online_users` sözlüğünden kaldırılır ve kalan tüm istemcilere hem kullanıcının ayrıldığı hem de güncellenen çevrimiçi kullanıcı listesi bildirilir.

```
@socketio.on('disconnect')
def handle_disconnect():
    username = online_users.pop(request.sid, None)
    if username:
        emit('user_left', {'username': username}, broadcast=True)
```

```
                emit('online_users',  list(online_users.values()),
broadcast=True)
```

Bir kullanıcı sohbet odasına girdiğinde bu olayı işler. Kullanıcı adı, oturum kimliğine eşleştirilerek bellekte saklanır ve tüm istemcilere yeni kullanıcı ve güncellenen çevrimiçi kullanıcı listesi bildirilir.

```
@socketio.on('join')
def handle_join(data):
    username = data['username']
    online_users[request.sid] = username
    emit('user_joined', {'username': username}, broadcast=True)
                emit('online_users',    list(online_users.values()),
broadcast=True)
```

Bir kullanıcı mesaj gönderdiğinde, bu mesaj doğrudan yayınlanmak yerine önce Google Cloud Pub/Sub'a iletilir. Ardından mesaj abone tarafından alınır, tüm kullanıcılara yayınlanır ve BigQuery'ye kaydedilir; böylece bulut iş akışının tamamı etkin bir şekilde kullanılır.

```
@socketio.on('message')
def handle_message(data):
    publish_message(data['username'], data['text'])
```

Analizi

Bu Endpoınt, BigQuery tablosuna yönelik analitik bir SQL sorgusu gerçekleştirir. Depolanan tüm mesajları kullanıcı adına göre gruplandırır ve her kullanıcı için iki metrik hesaplar: gönderilen toplam mesaj sayısı (message_count) ve mesaj başına ortalama kelime sayısı (avg_words). COALESCE fonksiyonu, word_count alanı boşsa, mesaj dizesini bölerek kelimeleri dinamik olarak saymaya geri dönülmesini sağlar.

Sonuçlar, mesaj sayısına göre azalan sırada sıralanır ve en aktif kullanıcılar belirlenir. Endpoint’i sonuçları bir JSON yanıtı olarak döndürür ve böylece herhangi bir frontend veya araçtan erişilebilir hale getirir.

```
@app.route('/stats')
def get_stats():
    query = """
```

```
        SELECT
            username,
            COUNT(*) as message_count,
             AVG(COALESCE(word_count, ARRAY_LENGTH(SPLIT(message, '
'))))) as avg_words
        FROM `eattheapple.chat_analytics.messages`
        GROUP BY username
        ORDER BY message_count DESC
    """
    results = bq_client.query(query).result()
    stats = []

    for row in results:
        stats.append({
            'username': row.username,
            'message_count': row.message_count,
            'avg_words': row.avg_words
        })
    return jsonify(stats)
```